EF CORE

# SaveChanges() Pipeline

A concise guide to what happens behind the scenes

**Main idea:** SaveChanges() is an orchestration process. It takes tracked changes in memory, turns them into database commands, executes them, and synchronizes the Change Tracker.

## 1. The pipeline

| Step | What happens? |
|---|---|
| **1. Detect Changes** | Compare original and current values; mark changed entities as Modified. |
| **2. Read States** | Inspect Added, Modified, Deleted, and Unchanged entities. |
| **3. Build Commands** | Prepare INSERT, UPDATE, or DELETE commands. |
| **4. Open Connection** | Open the database connection if it is not already open. |
| **5. Transaction** | Use a transaction when needed to keep the save operation consistent. |
| **6. Execute SQL** | Send commands to the database; the database engine executes them. |
| **7. Read Results** | Receive results and generated values, such as identity keys. |
| **8. Update States** | After success: Added/Modified → Unchanged; Deleted → Detached. |
| **9. Return Result** | Return the number of state entries written. |

## 2. Entity States

| State | Database operation | After successful save |
|---|---|---|
| Added | INSERT | Unchanged |
| Modified | UPDATE | Unchanged |
| Deleted | DELETE | Detached |
| Unchanged | No write command | Unchanged |

**Remember:** Changing a property changes the object in memory. It does not update the database until SaveChanges() succeeds.

## 3. Quick examples

### Update:

```
var student = context.Students.First(s => s.StudentId == 1);
student.PhoneNumber = "0799999999";
int entriesWritten = context.SaveChanges();
```

EF Core detects the change, generates UPDATE SQL, executes it, then marks the entity Unchanged.

### Insert:

```
var student = new Student { FirstName = "Ali", LastName = "Saleh" };
context.Students.Add(student);
context.SaveChanges();
```

EF Core generates INSERT SQL. The database may generate an identity key, which EF Core assigns to the object.

## 4. Important notes

• **Transactions:** Relational providers normally use transaction behavior for a SaveChanges operation when needed.

• **Errors:** Constraint violations, timeouts, and deadlocks can cause exceptions such as DbUpdateException.

• **Async:** SaveChangesAsync() uses asynchronous I/O where supported. It can improve scalability, but does not automatically make SQL execute faster.

• **Performance:** Investigate tracked entity count, SQL command count, network latency, indexes, locks, and transaction duration.

• **Avoid:** Calling SaveChanges() repeatedly inside loops when one save after the loop is appropriate.

**MENTAL MODEL**

**Tracked changes → Detect → Read states → Generate SQL → Execute → Update tracking → Return result**

**Key takeaway:** EF Core generates and sends SQL; the database executes it. Understanding this pipeline helps you reason about tracking, transactions, and performance.